

NavIce Antarctica: AI-Powered Maritime Navigation System

Smart India Hackathon (SIH) Prototype

NavIce is an end-to-end AI-powered maritime navigation system for the Antarctic region. It combines deep learning for iceberg trajectory forecasting (PyTorch GRU), a collision risk engine, and an A* pathfinding algorithm for safe polar route optimization.

The project features a brand-new **React frontend dashboard**, a **tactical 3D radar simulation**, and a robust **FastAPI backend**.

Features

- **Mission Control (React):** A beautifully designed frontend for managing routing, monitoring icebergs, and tracking the vessel's journey across the Southern Ocean.
 - **3D Tactical Radar (Three.js):** A fully interactive 3D simulation of a polar research vessel navigating iceberg-filled waters. Features dynamic collision avoidance, hysteresis-stabilized evasive maneuvers, and physics-based movement.
 - **Intelligent A* Route Optimizer:** Dynamically generated maritime routes that strictly adhere to a 6.1km safety standoff from all Natural Earth 50m landmasses and ice shelves.
 - **Deep Learning Forecasts:** Predicts iceberg drift trajectories 24h/48h into the future using a PyTorch GRU trained on the BYU/NIC Antarctic Iceberg Database.
 - **Environmental Data Integration:** Merges wind, sea state, and ocean current data to accurately evaluate risk and fuel consumption.
 - **Comprehensive API:** A FastAPI backend offering endpoints for iceberg tracking, weather forecasting, risk grids, and trajectory benchmarks.
-

Quick Start

1. Create Virtual Environment & Install Dependencies

(Assuming Windows PowerShell) `powershell python -m venv venv .\venv\Scripts\Activate`
`pip install -r requirements.txt`

2. Run the Data Preprocessing Pipeline

`powershell python src/preprocessing.py --raw_dir stats_database_v7.1 --output data/processed/iceberg_tracks_clean.csv --top_n 75`

3. Train the PyTorch GRU Model

`powershell python src/train_gru.py --epochs 10 --batch_size 128`

4. Evaluate Models & Generate Benchmark Plots

`powershell python src/evaluate.py`

5. Start the Integrated Web Dashboard (Backend + React UI)

The FastAPI application serves both the API and the React frontend on the same port. `powershell python -m uvicorn app.backend.main:app --host 0.0.0.0 --port 8000` Open your browser and navigate to **<http://localhost:8000>** to access the Mission Control dashboard.

Project Architecture

- **app/frontend/new_ui/**: The compiled React application, built with Vite, serving the main Mission Control dashboard.
- **app/frontend/radar_simulation.html**: The 3D Three.js tactical radar simulation that communicates directly with the backend route optimizer.
- **app/backend/main.py**: The FastAPI application serving all API endpoints and mounting the static frontend files.
- **src/route_optimizer.py**: The core A* routing logic utilizing geospatial collision detection against Antarctic coastlines.
- **src/train_gru.py**: The PyTorch model training pipeline for iceberg forecasting.
- **src/geospatial_obstacles.py**: Geographic validation engine ensuring safe offshore navigation and realistic roadstead snapping.

Indian Antarctic Stations Supported

Station	Coordinates	Region	Access
Princess Astrid (Maitri)	69.85S, 11.90E	Queen Maud Land	Coastal Staging
Bharati	69.40S, 76.18E	Larsemann Hills	Coastal Port

(Note: Maitri Base is strictly an inland facility. The application intelligently roots vessels to the coastal staging area on the Princess Astrid Coast.)

Model Benchmark (GRU vs Baseline)

Horizon	Model	Mean Error	Median Error
+24h	Constant Velocity	2.86 km	0.32 km
+24h	PyTorch GRU	2.25 km	0.17 km
+48h	Constant Velocity	5.16 km	0.95 km
+48h	PyTorch GRU	3.80 km	0.47 km